

Multi-server Password authenticated Key Exchange Protocol Based on MLWE

Yeming Yang
Information Engineering University
Zhengzhou, Henan, China
18888928280@163.com

Shuaichao Song
Information Engineering University
Zhengzhou, Henan, China
625262263@qq.com

Songhui Guo^{*}
Information Engineering University
Zhengzhou, Henan, China
songhui.guo@outlook.com

Abstract

Currently, PAKE (Password Authenticated Key Exchange) protocols on lattice using a single-server architecture are widely applied. However, such protocols are vulnerable to server leakage attacks, dictionary attacks, and other threats. To address these issues, researchers have proposed multi-server and two-server architecture-based PAKE protocols. However, PAKE protocols in a multi-server architecture require the use of complex cryptographic primitives such as signatures, and zero-knowledge proofs to ensure security, which reduces the execution efficiency of the protocol. To tackle these challenges, we propose two new multi-server password authentication key exchange protocols based on the MLWE (Module learning with errors) problem. Both protocols rely on MLWE instances, using Peikert's error coordination technique to enable two parties with similar values to compute the same result. Furthermore, we introduce the error pairing assumption and proves its security within random oracle model. The proposed protocol divides the password information into different shares and stores them on separate servers. In protocol 1, all servers and user collaboratively generate session keys, making it suitable for high-security application scenarios. In protocol 2, both user and servers generate session keys individually, which is ideal for high-efficiency application scenarios. Compared to similar protocols, both protocols lower computation and communication costs, better addressing practical application needs while providing protection against quantum computing attacks and server leakage threats.

CCS Concepts

• Security and privacy; • Security services; • Authentication;

Keywords

multi-server, MLWE, password authentication, key exchange

ACM Reference Format:

Yeming Yang, Shuaichao Song, and Songhui Guo. 2025. Multi-server Password authenticated Key Exchange Protocol Based on MLWE. In *2025 5th International Conference on Computer Network Security and Software Engineering (CNSSE 2025)*, February 21–23, 2025, Qingdao, China. ACM, New York, NY, USA, 14 pages. <https://doi.org/10.1145/3732365.3732394>

^{*}Corresponding author.



This work is licensed under a Creative Commons Attribution International 4.0 License.

CNSSE 2025, Qingdao, China

© 2025 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-1361-3/25/02

<https://doi.org/10.1145/3732365.3732394>

1 Introduction

With the rapid development of information technology in the new era, the Password Authenticated Key Exchange (PAKE) protocol, which is easy to use and does not require additional public key infrastructure, has become the most widely used Internet security protocol at present [1]. The PAKE protocol enables communication parties to authenticate each other over the open channel and negotiate the same session key by means of easy-to-remember passwords. The PAKE protocol enables communicating parties to authenticate each other on an open channel and negotiate the same session key through an easy-to-remember password.

Currently, most PAKE protocols are built on traditional difficult problems, such as the discrete logarithm problem and integer factorization problem [2-3]. However, with the development of quantum computing, PAKE protocols based on these traditional difficult problems are vulnerable to attacks by quantum computers. As a result, research on PAKE protocol that can withstand quantum computer attacks has become a current hot spot [4-5]. Among the cryptographic schemes resistant to quantum computing attacks, cryptographic schemes on lattice are favored due to their parallelism, efficiency, simplicity and versatility [6-7]. Therefore, cryptographic schemes on lattice are considered among the most promising post-quantum cryptography solutions for the post-quantum era [8].

For widely used PAKE protocols on lattices, user's passwords are typically stored in plaintext or as hash values on a server [9-11]. These protocols are not well-researched and suffer from low communication efficiency, with most relying on a single-server setup. If a server is compromised, there is a risk of user data leakage. To address these issues, researchers have proposed password authentication protocols based on two-server and multi-server architectures, which can resist server leakage attacks. Therefore, designing a secure and efficient lattice-based PAKE protocol in two or multi-server environments is of great significance.

To date, only a few multi-server PAKE protocols based on public key infrastructure (PKI) models for lattice-based systems exist, and these cannot be applied to two-server scenarios [5]. Moreover, multi-server architectures necessitate the use of cryptographic primitives such as digital signatures and zero-knowledge proof to ensure security, significantly increasing computational overhead [12-17]. Lattice-based multi-server PAKE protocols also require costly cryptographic primitives such as fully homomorphic encryption and secret sharing, which decrease protocol efficiency and increase communication and storage costs. On the other hand, due to the lower security of the PAKE protocol in a two-server architecture compared to that in a multi-server architecture, and its poor

practical application and deployability, we primarily focus on the multi-server PAKE protocol [18–20].

Therefore, to address the problem that existing protocols cannot resist server leakage attacks and consume a large amount of storage resources in multi-server architectures, we focus on provably secure password-only multi-server PAKE protocols to improve the deployability of the protocols as well as the communication efficiency. The experimental results show that the proposed PAKE protocol has high execution efficiency and security compared with other related schemes.

2 RELATED WORK

In 2003, Brainard et al in [21], first proposed a novel two-server Password Authenticated Key Exchange (PAKE) protocol. This protocol maintains security even if only one server is compromised. However, it lacks security proofs. Subsequently, Katz et al in [22] proposed a password-only two-server PAKE protocol, which was the first two-server protocol to undergo security analysis in the standard model. Despite its advancements, the two-server architecture results in higher server overheads, leading to inefficiencies in practical applications. In 2006, Yang et al in [23] improved upon previous protocols, enhancing the security of the session key. However, this improvement resulted in an increase in the number of communication rounds required by the protocol to ten.

In order to improve the communication efficiency Jin et al. [24] proposed a password-only two-server Password-Authenticated Key Exchange (PAKE) protocol. This protocol does not introduce additional cryptographic primitives and requires only six rounds of communication to achieve mutual authentication, significantly reducing the overall number of communication rounds. In 2013, Yi et al. [25] developed a PAKE protocol based on ID identifiers. In this protocol, the client must remember the passwords for both servers, as well as their IDs. Consequently, this adds to the memorization burden on the client. Kiefer et al. [26] introduced the concept of a distributed smooth projected hash function and presented a framework for a dual-server PAKE protocol using this function. However, this protocol incurs high computational overhead due to the extensive use of numerous cryptographic primitives. Kumari et al. in [27], proposed a three-dimensional two-server PAKE protocol designed to handle passwords. This protocol became the first two-server PAKE protocol proven to resist offline dictionary attacks. In 2018, Chattaraj et al. [28] proposed a two-server authentication protocol, which provides security proofs within random oracle model. This ensures that the protocol is resistant to replay and man-in-the-middle attacks while maintaining a balance between computational overhead and security properties.

However, the two-server architecture has certain limitations that have led to criticism regarding its security and practical deployment capabilities. Consequently, researchers have begun to shift their focus towards PAKE protocols that utilize the multi-server architecture.

In the realm of multi-server PAKE protocols, the first multi-server protocol was introduced by Ford et al. in 2000 [29]. They proposed a threshold PAKE protocol based on a Public Key Infrastructure model, where multiple servers (n servers) collaborate to share the user's password, authenticate the user, and establish a

shared session key with them. For security, the protocol remains secure as long as no more than $n-1$ servers are compromised. However, because the password is distributed among multiple servers, this significantly increases storage overhead. Jablon et al in [30], later modified this approach by proposing a PAKE protocol in a password-only setup. Nonetheless, neither of these two protocols has been rigorously proven secure enough to effectively resist server leakage attacks. In subsequent research on PAKE protocols, Mackenzie et al in [31], developed a t -out-of- n threshold PAKE protocol. This protocol requires only t out of the n servers to collaborate for user authentication, and it can be deemed secure as long as no more than $t-1$ servers are attacked. The security of this protocol is evaluated under the BPR model.

The above protocols are all designed based on traditional difficult problems. With the development of quantum computing and quantum algorithms, PAKE protocols based on traditional difficult problems such as discrete logarithms and integer factorization face significant security threats. PAKE protocols based on lattice theory have gained attention in post-quantum cryptography due to their high efficiency and strong security properties.

PAKE protocols based on lattice have received less attention compared to those based on traditional hard problems. In 2009, Katz et al in [32], constructed a lattice-based approximate smooth projective hash function, which laid the groundwork for the first on-lattice PAKE protocol. To address server leakage attacks and known quantum attacks on a single server, Yang et al in [33], developed the first 3PAKE(three-party password authenticated key exchange) protocol based on authentication elements and the RLWE(ring learning with errors) problem. In this framework, the server only stores the authentication element generated from information such as the password, without retaining the actual password itself. It means that even if server information is compromised, an attacker cannot use the leaked authentication element to deduce the user's passphrase, effectively mitigating the risk associated with server information leakage. In 2020, Zhang et al in [34], pointed out that Yang et al.'s protocol struggled to withstand offline dictionary attacks, leading them to propose a new 3PAKE protocol based on authentication elements and SPHF (smooth projective hash function) that offers enhanced security without incurring additional communication overhead. Additionally, in 2020, Shu et al in [35], adapted the concept of verifier and designed a three-party PAKE protocol on ideal lattices, aiming to resist server leakage attacks. The security of Shu et al.'s protocol was analyzed within the composable security model.

Due to the limitations of a single-server architecture, if an attacker successfully accesses data from the server, they can exploit stored password information or intermediate keys to launch attacks, which could compromise the entire security of the key exchange process. Consequently, the overall security of the single-server architecture is significantly diminished.

The two-server PAKE protocol is an effective solution for resisting server leakage attacks. However, most existing two-server PAKE protocols rely on public key infrastructure, making them less secure compared to PAKE protocols based on multi-server architecture. Additionally, these two-server protocols are often less adaptable and deployable in real-world application environments. While there are some threshold PAKE protocols designed

for multi-server architectures, their complexities pose challenges. These protocols do not utilize a password-only setting and require additional cryptographic functions such as signature checks, zero-knowledge proofs, and other costly cryptographic primitives to ensure security. This reliance significantly increases computational overhead, resulting in substantial limitations, low efficiency, and challenges with widespread deployment and application.

To address the issues mentioned above, multi-server password-authenticated key exchange protocols based on lattice have become the focus of current research. However, this area of study has seen limited investment and development until recently. In 2024, Chen et al. in [36], proposed a distributed smoothed projected hash function based on lattice that does not rely on cryptographic primitives such as signatures or secret sharing. This proposal utilized the LWE (Learning with Errors) problem and introduced a more efficient security model for multi-server PAKE. Despite these advancements, the protocol's reliance on the smoothed projected hash function and extensive matrix operations results in significant computational demands. It can lead to excessive computational loads for resource-constrained users, increasing the overall computational overhead. Consequently, the protocol faces challenges in being widely adopted in practical applications and cannot be deployed reasonably in many scenarios.

In summary, to address the challenges of quantum computing attacks and server leakage incidents, we propose two secure and efficient PAKE protocols. These protocols are based on the MLWE (Module Learning with Error) problem and are designed for multi-server architectures, suitable for high-performance and high-security application scenarios [37]. This work draws upon research ideas presented in the literature [31, 36–37], and the main contributions are outlined as follows.

1. **Design a highly secure PAKE protocol that can resist server attacks.** The protocol is based on a multi-server architecture that divides the user's password information into several parts, storing each part separately across multiple servers. A gateway server is set up to facilitate interactive authentication. Together, these servers collaborate to generate a final session key. Even if an attacker compromises up to $N-1$ servers, the protocol maintains security and minimizes the risks associated with server leaks.

2. **Design a high-performance PAKE protocol that is resistant to server leakage attacks.** In this protocol, each server and user independently generate a session key to ensure security. This approach is similar to single-server architecture protocols, significantly enhancing the communication efficiency of the protocol. It is particularly suitable for application environments that require moderate security combined with high efficiency.

3 PRELIMINARIES

In this section, the definition of MLWE problems and the notation used in this paper are presented.

3.1 Centered Binomial Distribution Sampling

The central binomial distribution sampling on the lattice can be used to improve the sampling efficiency and ensure that the results are indistinguishable from the discrete Gaussian distribution sampling [38].

Definition 1: CBD Algorithm

β_η is an approximate Gaussian distribution with zero expectation and the standard deviation is $\beta = \sqrt{\eta/2}$. When sampling, we randomly select $a_i, b_j \in \{0, 1\}$ and compute the sampling value $\sum_{i=0}^{\eta} (a_i - b_i)$, and the final sampled result is an integer within the range $[-\eta, \eta]$.

3.2 Modular Learning with Errors (MLWE) Problem

Due to the potential attack threats arising from the algebraic structure of RLWE, Langlois et al. in [37], proposed MLWE problem in 2015. MLWE problem replaces integer ring parameters in the LWE problem matrix with ring polynomials from RLWE problem, and introduces a tiny dimension in the polynomial ring structure to control the size of the public key polynomial matrix. The number of dimensions makes the operating efficiency comparable to RLWE while ensuring the same security. Compared to LWE and RLWE, the MLWE problem has a more complex algebraic structure, which is not vulnerable to the weaknesses of ideal lattice structures.

Definition 2: Decision Module Learning with Errors (DMLWE)

Define n, q, k are positive integers, where n is the dimension of a vector, q is the modulus in the polynomial ring $R_q^k = \mathbb{Z}_q[x]/(x^n + 1)$ and k is the dimension of a polynomial matrix. β_η^k is a central binomial distribution on R_q^k . By randomly selecting the polynomial matrix $A \rightarrow R_q^{k \times k}$ and uniformly selecting the secrets $\rightarrow \beta_\eta^k$ and the error vector $e \rightarrow \beta_\eta^k$. The Decision MLWE problem is to determine whether $b = As + e$ or is chosen uniformly at random.

3.3 Reconciliation Mechanism

Lattice-based problems involve random error term that are essential for ensuring security in a post-quantum computing environment. However, these error terms not only make it challenging to solve lattice-based problems but also prevent both parties in a communication from obtaining precisely the same session key.

To address this issue, in 2012, Ding proposed the concept of error coordination and referred to it as the Ding-style error coordination mechanism. This mechanism enables users with similar values to extract the same bits through a coordination function. In later research, Peikert pointed out a flaw in the Ding-style error coordination mechanism: the bits extracted by both parties in the protocol are not uniformly distributed [39]. Peikert proposed a novel improvement, called the Peikert-style error coordination mechanism. The mechanism is described as follows: supposing q is a prime number greater than 2, define $\mathbb{Z}_q = \{-q/2, \dots, 0, \dots, q/2 - 1\}$.

Define the following three intervals: $I_0 = \{0, 1, \dots, q/4 - 1\}$, $I_1 = \{-q/4, \dots, -1\}$, $E = [-q/8, q/8] \cap \mathbb{Z}$.

Definition 3: The cross-rounding function: $\langle x \rangle_{q,2} = \lfloor 4/q \cdot x \rfloor \bmod 2$;

Definition 4: The modular rounding function: $\lfloor x \rfloor_{q,2} = \lfloor 2/q \cdot x \rfloor$;

Definition 5: The reconciliation function: $rec(w, \sigma) = \begin{cases} 0, & w \in I_\sigma + E \bmod q, \\ 1, & \text{else} \end{cases}$;

Lemma 1: For an even modulus q , if $w \in \mathbb{Z}_q$, $e \in E$, $w + v = e \bmod q$, then have $\lfloor v \rfloor_{q,2} = rec(w, \langle v \rangle_{q,2})$.

Lemma 2: For an odd q , define a random function $dbl(\cdot) : \mathbb{Z}_q \rightarrow \mathbb{Z}_{2q}$, $dbl(x) = 2x - \bar{e}$ where $\bar{e}$ is a random term. The probability that $\bar{e}$ is equal to 0 is $1/2$, and the probability that $\bar{e}$ is equal to -1 or 1 is $1/4$. If $v \in \mathbb{Z}_q$ is uniformly random, let $\bar{v} = dbl(v) \in \mathbb{Z}_{2q}$, then $[\bar{v}]_{2q,2}$ on $\mathbb{Z}_{2q}$ is also uniformly random. if $w + v = 2e \bmod 2q$ is known, and $w \in \mathbb{Z}_{2q}$, $e \in E$, then have $[\bar{v}]_{2q,2} = rec(w, \langle \bar{v} \rangle_{2q,2})$.

3.4 PWE assumption based on MLWE

Error Pairing Assumption is a method proposed to aid in constructing security proofs for PAKE protocols. In this paper, we specifically reference the PWE (Polynomial Learning with Errors) assumption developed by Ding et al. in [40], which is based on the RLWE problem. By enhancing the PWE assumption proposed by Ren and Guo et al. in [41-42], we introduce a new PWE assumption grounded in the MLWE problem, utilizing Peikert's error coordination mechanism.

Definition 6: Error Pairing Assumption

Define the adversary be an algorithm in probabilistic polynomial events, and the input is (A, X, Y, σ) , where $A \leftarrow R_q^{k \times k}$, $X \leftarrow R_q^k$, $s_y, e_y, e_z \leftarrow \beta_\eta^k$, $Y = A^T s_y + e_y \in R_q^k$, $\sigma = dbl(X^T s_y + e_z)_{2q,2} \in \{0, 1\}^n$, $K = dbl(X^T s_y + e_z)_{2q,2} \in \{0, 1\}^n$. Then the goal of the adversary C is to obtain the value of the K from the input of (A, X, Y, σ) . In this paper, the adversary's advantage in breaking the PWE assumption based on MLWE problem is formally defined as follows:

$$Adv_{\Pi}^{AKE}(\mathcal{A}) \leq Adv_{R_q}^{PWE}(\mathcal{A}) = \Pr[K \in \mathcal{A}(A, X, Y, \sigma)] \quad (1)$$

Let $Adv_{R_q}^{PWE}(t, N) = \max_C \{Adv_{R_q}^{PWE}(C)\}$, where, all adversaries with the maximum time complexity t will take advantage of the maximum attack, and these adversaries will output a list containing at most N elements belonging to $\{0, 1\}^n$. The MPWE assumption shows that t is negligible for t and N under the bounds of security parameters.

This article describes the simplification process from the PWE assumption to the decision MLWE problem.

Definition 7: Decision PWE problem

Given $(A, X, Y, \sigma, K) \in R_q^{k \times k} \times R_q^k \times R_q^k \times \{0, 1\}^n \times \{0, 1\}^n$, where $Z = X^T s_y + e_z$, $\sigma = dbl(Z)_{2q,2} = dbl(X^T s_y + e_z)_{2q,2}$, $K = rec(2X^T s_y, \sigma)_{2q,2}$. Let $s_y, e_y \leftarrow \beta_\eta^k$, $e_z \leftarrow \beta_\eta$, then the DPWE problem is to decide (Y, Z) whether $Y = A^T s_y + e_y$ and $Z = X^T s_y + e_z$ or randomly uniformly generated $(Y, Z) \in R_q^k \times R_q$ can be distinguished. If DMPWE is hard, PWE is hard.

Definition 8: MLWE-DH problem

Given $(A, X, Y, Z) \in R_q^{k \times k} \times R_q^k \times R_q^k \times R_q$, where $Y \in R_q^k$, $Z = X^T s_y + e_z$. Let $e_y \leftarrow \beta_\eta^k$, $e_z \leftarrow \beta_\eta$, MLWE-DH problem is to decide whether $Y = A^T s_y + e_y$ and $Z = X^T s_y + e_z$ or random uniformly generated $(Y, Z) \in R_q^k \times R_q$ can be distinguished.

Theorem 1: Assuming that the MLWE-DH problem is hard, the DPWE problem is also hard.

Proof of Theorem 1. Suppose an algorithm D exists that can solve the DPWE problem with a non-negligible advantage. By using algorithm D , a distinguisher D' can be constructed to solve the MLWE-DH problem as follows:

- Input (A, X, Y, Z) .

- Compute $\sigma = dbl(Z)_{2q,2}$, $K = rec(2X^T s_y, \sigma)_{2q,2}$.
- Call D to solve DPWE problem using the input (A, X, Y, σ, K) .
- If D outputs 1 then D' outputs 1, means that $Y = A^T s_y + e_y$, $Z = X^T s_y + e_z$. otherwise, $(Y, Z) \in R_q^k \times R_q$.

It can be seen from Theorem 1 that the MLWE-DH problem is difficult, which is contrary to the solvability of the MLWE-DH problem. Therefore, if the MLWE-DH problem is hard, then DPWE is also hard.

Theorem 2: Assuming that the decision MLWE problem is hard, the MLWE-DH problem is also hard.

Proof of Theorem 2. Suppose there exists an algorithm D that can solve the MLWE-DH problem with a non-negligible advantage. Then a distinguisher D' can be constructed to solve the decision MLWE problem based on the algorithm. Specifically, two decision MLWE instances (A_1, b_1) and (A_2, b_2) with the same private key $s_y \in R_q^k$ are first given. The execution process of a distinguisher D' is as follows: Given two samples of an MLWE challenge (A_1, b_1) and (A_2, b_2) , both share the same $s_y \leftarrow R_q^k$. We call algorithm D to build a distinguisher D' to solve the MLWE problem as follows:

- Set $(A, X, Y, Z) = (A_1, A_2, b_1, b_2)$;
- Input (A, X, Y, Z) to D .
- If D outputs 1, then D' outputs 1, means $b_1 \leftarrow A_1^T s_y + e_y$, $b_2 = A_2^T s_y + e_z$ for $e_z \leftarrow \beta_\eta$; otherwise, D outputs 0, then D' outputs 0, means that $b_1, b_2 \in R_q^k \times R_q$.

Suppose an algorithm can solve the MLWE-DH problem with non-negligible advantage. In that case, we can construct an algorithm to solve the decision MLWE problem with non-negligible advantage. However, this is contrary to the hardness of the decision MLWE problem itself, so if the decision MLWE problem is hard, the MLWE-DH problem is also hard.

In summary, it can be concluded that solving the PWE problem is difficult.

4 Security model

This section mainly adopts BPR literature models in the literature to give a more realistic multi-server PAKE security model, which can more accurately evaluate the real risk faced by the multi-server PAKE protocol. Security game: an algorithm game between the challenger C and the adversary $\mathcal{A}$. The challenger C runs the protocol P on behalf of the honest user and make an instance of the simulator simulating the protocol.

Execute($\Pi_U^x, \Pi_{S_i}^x, \Pi_{S_k}^x$) **query:** This query characterizes the passive attack capability of the adversary $\mathcal{A}$. The adversary $\mathcal{A}$ can obtain the information transmitted by the participants on the channel by querying the instances

Send(Π_p^x, m) **query:** The adversary $\mathcal{A}$ can imitate the participants to send a message and interact with the instance Π_p^x . The adversary sends the information to the instance of participant P . After receiving it, the instance executes according to the protocol specification and returns a response message to the adversary $\mathcal{A}$.

Reveal(Π_U^x) **query:** The adversary $\mathcal{A}$ obtains the session key SK_U^x in the user instance through this query.

Corrupt(Π_p^x) **query:** The adversary $\mathcal{A}$ obtains the password of user through this query.

Table 1: Symbol definitions

Symbols	Definitions
U	User
S_i, S_k	Server
TID	Temporary identity
n	Random number
A	Common parameters
s, e	Secret, error value
$Auth$	Authentication information
sk	Session key
$H_0(\cdot), H_1(\cdot), H_2(\cdot)$	$\{0, 1\}^* \rightarrow \{0, 1\}^l$
w	Doubling value
σ	Signal value
c	Coordination value

Test(Π_P^x) query: This query characterizes the adversary's ability to distinguish a real session key from random values. Randomly flip a coin $b \in \{0, 1\}$, if $b = 0$, return the real negotiated key, if $b = 1$, return any random number in the session key space. Finally, the adversary $\mathcal{A}$ outputs b' , if $b' = b$, the adversary $\mathcal{A}$ wins.

Definition 9: Partnership

For the user instance Π_U^x , we assume $Sid_U^x = (Sid_{U,Sp}^x, Sid_{U,Sk}^x)$. For the server instance Π_{Si}^y , we assume $Sid_{Si}^y = (Sid_{Si,U}^y, Sid_{Si,Sk}^y)$. If user U and server S_k are in a partnership, $Sid_{U,Si}^x = Sid_{Si,U}^y$ when $U \in Pid_{Sp}^y, Si \in Pid_U^x$. If server S_i and S_k are in a partnership, $Sid_{Si,Sk}^y = Sid_{Sk,Si}^z$ when $S_k \in Pid_{Sp}^y, S_i \in Pid_{Sk}^z$.

Definition 10: Semantic safety

In the execution of the agreement, the adversary $\mathcal{A}$ can perform multiple queries such as $Execute(\Pi_P^x) \setminus Send(\Pi_P^x, m)$ during the game. However, for freshness instances, the query $Test(\Pi_P^i)$ is executed only once, and the corresponding bits b' are output. If $b = b'$, the adversary $\mathcal{A}$ wins; if $b \neq b'$, $\mathcal{A}$ fails. The probability of the adversary winning is defined as: $Adv_{\Pi}^{AKE}(\mathcal{A}) = |2Pr(b = b') - 1|$. If the probability of the adversary winning is negligible, then the protocol is AKE secure.

5 OUR PROTOCOLS

To accommodate varying security and performance requirements across different application scenarios, we establish PAKE protocol schemes within a multi-server framework and divides them into two distinct designs. We first give the symbol definitions in Table 1 that will be utilized in the subsequent sections.

5.1 Protocol 1

In this protocol, all servers and user jointly generate a session key.

5.1.1 System initialization process. (1) $U \rightarrow S_i, S_k: (ID_U, A, PW_{Si}, PW_{Sk})$.

When the user U joins the protocol system, the user U sends registration information to the servers S_i, S_k ($1 \leq k \leq N, i \in k$). The identity information ID_U and password PW_U are chosen randomly for each user U and we assume that this password can be mapped in a one-to-one fashion to Z_q . The user U randomly selects $A \leftarrow$

$R_q^{k \times k}$. If S_i and S_k are the servers associated with the user U , then password shares $PW_{Si}, PW_{Sk} \in Z_q$ are chosen uniformly at random subject to $\sum_{k=1}^N PW_U^k \pmod{q} = PW_U$, with PW_{Si} given to server S_i and PW_{Sk} given to server S_k .

(2) $S_i \rightarrow U: (TID_{Ui})$.

After receiving U 's registration information, S_i adds $\langle ID_U, PW_{Si} \rangle$ to its database. We use a random number n_0 and the identity information ID_U to compute a temporary identity $TID_{Ui} = H_0(ID_U || n_0)$ for user U . Server S_i sends TID_{Ui} to U .

(3) $S_k \rightarrow U: (TID_{Uk})$.

After receiving U 's registration information, S_k adds $\langle ID_U, PW_{Sk} \rangle$ to its database. We use a random number n_1 and the identity information ID_U to compute $TID_{Uk} = H_0(ID_U || n_1)$ for U . And Server S_k save TID_{Uk} and send TID_{Uk} to user U .

5.1.2 Authentication and key exchange phase. When user U needs to share the session key for communication, it is essential to establish two-way authentication between users U with the server S_i and S_k . It facilitates the exchange of shared session keys. The authentication and key exchange process is shown in Figure 1.

1. $U \rightarrow S_i: M_0 = (TID_{Ui}, n_2, b_U)$.

The user U samples $(s_U, e_U) \leftarrow \beta_\eta^k \times \beta_\eta^k$ uniformly, generates random number n_2 , and calculates $b_U = As_U + e_U$. The user U sends $M_0 = (TID_{Ui}, n_2, b_U)$ to the server S_i .

2. $S_i \rightarrow S_k: M_1 = (TID_{Ui}, n_2, n_3, b_U, b_{Si})$.

Upon receiving the request, the server confirms the identity of the user applying for authentication and key exchange through TID_{Ui} . Then the server S_i samples $(s_{Si}, e_{Si}, e_i) \leftarrow \beta_\eta^k \times \beta_\eta^k \times \beta_\eta^k$ and $\bar{e}_i \leftarrow \beta_n$ uniformly, generates random number n_3 and calculates $b_{Si} = A^T s_{Si} + e_{Si}$. The server S_i sends $M_1 = (TID_{Ui}, n_2, n_3, b_U, b_{Si})$ to the server S_k .

3. $S_k \rightarrow S_i: M_2 = (TID_{Uk}, Auth_{Uk}, Auth_{ik}, n_{k4}, n_{k5}, \sigma_k, \sigma_{k1}, b_{Sk})$.

After the server S_k receives the message M_1 , the server S_k samples $(s_{Sk}, e_{Sk}) \leftarrow \beta_n^k \times \beta_n^k$ and $(\bar{e}_{k1}, \bar{e}_{k2}, e_{sk1}, e_{sk2}) \leftarrow \beta_n \times \beta_n \times \beta_n \times \beta_n$, uniformly, generates random numbers n_{k4}, n_{k5} , and calculates $b_{Sk} = A^T s_{Sk} + e_{Sk}$, $w_k = b_U^T s_{Sk} + e_{sk1}$, $w_{k1} = b_{Si}^T s_{Sk} + e_{sk2}$. Next, The server S_k uses the randomized function, cross-rounding function and modular rounding function to calculate $\bar{w}_k \leftarrow dbl(w_k)$, $\sigma_k \leftarrow \langle \bar{w}_k \rangle_{2q,2}$, $c_k \leftarrow \lfloor \bar{w}_k \rfloor_{2q,2}$ and $\bar{w}_{k1} \leftarrow dbl(w_{k1})$, $\sigma_{k1} \leftarrow \langle \bar{w}_{k1} \rangle_{2q,2}$, $c_{k1} \leftarrow \lfloor \bar{w}_{k1} \rfloor_{2q,2}$. The server S_k calculates $Auth_{U,k} = H_1(TID_{Uk} || n_2 || n_{k4} || c_k || b_U || b_{Sk} || PW_{Sk})$ between user and server S_k . Then it calculates $Auth_{i,k} = H_1(TID_{Ui} || TID_{Uk} || n_3 || n_{k5} || c_{k1} || b_{Si} || b_{Sk})$ between the server S_i and S_k . The server S_k calculates $SK_{Si,Sk} = H_2(TID_{Ui} || TID_{Uk} || n_3 || n_{k5} || c_{k1} || b_{Si} || b_{Sk})$ between server S_i and S_k . It sets the shared key $sk_{U,Sk} = H_2(TID_{Uk} || n_2 || n_{k4} || c_k || b_U || b_{Sk})$ between the user U and the server S_k . After completing $N-1$ calculations, we arrive at the final result $SK = \sum_{k=1}^{N-1} SK_{sk,U}$. The server S_k sends M_2 to the server S_i .

4. $S_i \rightarrow U: M_3 = (Auth_{U,Sk}, Auth_{Si,U}, \sigma_k, \sigma_{i2}, n_3, n_{k4}, b_{Si}, b_{Sk})$.

After the server S_i receives the message M_2 , the server S_i calculates $w_{i2} = b_U^T s_{Si} + \bar{e}_i$. Then the server S_i uses the randomized function to calculate $\bar{w}_{i2} \leftarrow dbl(w_{i2})$; uses the cross



Figure 1: Authentication and key exchange phase

rounding function to calculate $\sigma_{i2} \leftarrow \langle \bar{w}_{i2} \rangle_{2q,2}$; uses the modular rounding function to calculate $c_{k2} \leftarrow \lfloor \bar{w}_{k2} \rfloor_{2q,2}$. The server S_i calculates $w_{i'} = b_{SK}^T s_U$, then uses Peikert error reconciliation function to calculate $c_{i2'} = \text{rec}(2w_{i2'}, \sigma_{i2})$. Next, the server calculates the verification information $\text{Auth}_{i,k'} = H_1(\text{TID}_{U,i} || \text{TID}_{U,k} || n_3 || n_{k5} || c_{k1'} || b_{Si} || b_{Sk})$ with the server S_k . If $\text{Auth}_{i,k'} = \text{Auth}_{i,k}$, the server continues to calculate $SK_{Si,Sk} = H_2(\text{TID}_{U,i} || \text{TID}_{U,k} || n_3 || n_{k5} || c_{k1'} || b_{Si} || b_{Sk})$ with the server S_k . Otherwise, the protocol shall terminate. Next, The Server calculates $\text{Auth}_{Si,U} = H_1(\text{TID}_{U,i} || n_2 || n_3 || c_{i2} || b_U || b_{Si} || PW_{Si})$, and the shared key $sk_{U,Si} = H_2(\text{TID}_{U,i} || n_2 || n_3 || c_{i2} || b_U || b_{Si})$ with the user. The server S_1 sends $M_3 = (\text{Auth}_0, \text{Auth}_2, \sigma_0, \sigma_2, n_3, n_4, b_{S1}, b_{S2})$ to the user U .

5. $U : SK$.

After receiving the message M_3 , the user U calculates $w_{i2'} = b_{Si}^T s_U$ and applies the Peikert error reconciliation function to calculate $c'_{i2} = \text{rec}(2w'_{i2}, \sigma_{i2})$. Then the user U calculates $\text{Auth}'_{Si,U} = H_1(\text{TID}_{U,i} || n_2 || n_3 || c'_{i2} || b_U || b_{Si} || PW_{Si})$, and compares it with the received authentication information $\text{Auth}_{Si,U}$ of the server S_i , if $\text{Auth}_{Si,U'} = \text{Auth}_{Si,U}$, we can ensure the server's S_i identity. Next, the user U calculates the shared key $sk_{U,Si} = H_2(\text{TID}_{U,i} || n_2 || n_3 || c'_{i2} || b_U || b_{Si})$ with the server. The user U continues to calculate $w_{k'} = b_{SK}^T s_U$ and applies the Peikert error reconciliation function to calculate $c'_k = \text{rec}(2w'_k, \sigma_k)$. Then the user calculates $\text{Auth}'_{U,Sk} = H_1(\text{TID}_{U,k} || n_2 || n_{k4} || c'_k || b_U || b_{Sk} || PW_{Sk})$ and compares it with Auth_1 . If $\text{Auth}'_1 = \text{Auth}_1$, the server S_k 's identity is credible, and user U obtains the same session key $sk_{U,Sk} = H_2(\text{TID}_{U,k} || n_3 || n_{k5} || c'_k || b_U || b_{Sk})$ with the server S_k . After completing $N-1$ calculations, we arrive at the final result $SK = \sum_{k=1}^{N-1} SK_{Sk,U}$.

5.1.3 Security analysis. This section provides a security proof of the PAKE protocol proposed in this paper based on the BPR model. Let Π be the protocol proposed in this paper. The adversary $\mathcal{A}$ represents an attack protocol's probabilistic polynomial-time adversary, and assume that a simulator C runs the protocol. Suppose the adversary $\mathcal{A}$ selects one of the servers to attack. The proof process assumes that the number of servers compromised by the adversary is less than or equal to $(N-1)$, and all of its servers except the gateway server can be compromised by the adversary $\mathcal{A}$.

Theorem 3: Let Π be the protocol proposed in this paper, and L be the dictionary library of size $|L|$. $q_{ex}, q_{se}, q_{re}, q_{co}$ respectively represent Execute, Send, Reveal, Corrupt queries, and q_{ro} represent querying random oracles. Assume that the adversary $\mathcal{A}$ can control the simulator access to all oracles that. The simulator runs the protocol Π . For the adversary $\mathcal{A}$, the advantage of its attack protocol is defined:

$$\begin{aligned} Adv_{\Pi}^{AKE}(\mathcal{A}) \leq & (q_{ex} + q_{se} + q_{ro})(q_{ex} + q_{se}) \left/ q^{nk} + q_{ro} \right/ q^{nk} + q_{se}/2^d \\ & + 4Adv_{R_q^k}^{DMLWE}(t, q_{ro}) + 4Adv_{R_q^k}^{PWE}(t, q_{ro}) + q_{se}/L \end{aligned} \quad (2)$$

Proof. The game $G_i (i = 0, \dots, 6)$ is defined in the security model of this section. $G_0 = \Pi$ corresponds to the honest implementation of the protocol. G_6 involves simulating the protocol in an ideal situation under the random oracle model, during which the adversary's advantage in attacking the protocol is negligible. The

advantage of the adversary attacking the protocol in G_i is greater than that of the adversary attacking the protocol in G_{i-1} , the security of the protocol is gradually reduced, we can conclude the following:

$$\begin{aligned} Adv_{G_0}^{AKE}(A) & \leq Adv_{G_1}^{AKE}(A) + \varepsilon_1 \leq Adv_{G_2}^{AKE}(A) \\ & + \varepsilon_2 \leq Adv_{G_3}^{AKE}(A) + \varepsilon_3 \leq Adv_{G_4}^{AKE}(A) + \varepsilon_4 \\ & \leq Adv_{G_5}^{AKE}(A) + \varepsilon_5 \leq Adv_{G_6}^{AKE}(A) + \varepsilon_6 \end{aligned} \quad (3)$$

The $\varepsilon_1 \dots \varepsilon_6$ in the formula is negligible. By combining these negligible values with the probability of success of an online password-guessing attack, we can calculate the adversary's advantage of the success in attacking the protocol.

G_0 : This game simulates a real attack under the random oracle model in this paper. The advantage of an adversary in breaking the protocol can be defined as: $Adv_{G_0}^{AKE}(A) = 2\Pr[\text{Succo}] - 1$.

G_1 : We simulates a random oracle $H_i(\cdot) (i \in \{0, 1\})$ by maintaining hash lists $\wedge H_0$ and $\wedge H_1$. If there is a record (m, l) in the hash list, l is returned; otherwise, an element $l \in Z_q$ is randomly selected, (m, l) is added to the list, and l is returned.

Lemma 3: For probabilistic polynomial adversaries $\mathcal{A}$, G_1 and G_0 are indistinguishable.

Proof: Obviously, if adversary $\mathcal{A}$ can break hash function, adversary $\mathcal{A}$ can distinguish the output of the hash function and random string. Thus, G_1 and G_0 are indistinguishable: $|Adv_{G_0}^{AKE}(\mathcal{A}) - Adv_{G_1}^{AKE}(\mathcal{A})| \leq \text{negl}$.

G_2 : The simulator excludes session information where collisions have occurred. After performing queries such as *Execute*, *Send* and random oracle queries, b_U, b_{Si}, b_{Sk} may have been used in previous queries, the protocol is terminated.

Lemma 4: For probabilistic polynomial adversaries $\mathcal{A}$, the advantage of distinguishing between games G_2 and G_1 games is: $|Adv_{G_2}^{AKE}(\mathcal{A}) - Adv_{G_1}^{AKE}(\mathcal{A})| \leq (q_{ex} + q_{se} + q_{ro})(q_{ex} + q_{se})/q^{nk}$.

Proof: The adversary can make at most q_{se}, q_{ex}, q_{ro} queries and other random oracle queries. All of these queries correspond to instances of the MLWE problem. The elements in the ring R_q^k coefficient is k , and the dimension is n . Each coefficient can take q different values, so the ring elements in total have at most q^{nk} possible combinations. The values of the ring elements generated in the current query might coincide with those from previous queries, such as *Execute*, *Send* queries. If no collision occurs, there must still be $q_{ex} + q_{se}$ distinct values. In other words, since solving the MLWE problem is difficult, obtaining the same values of b_U, b_{Si}, b_{Sk} is also difficult. For polynomial-time adversary $\mathcal{A}$, the probability of distinguishing G_2 from G_1 is: $|Adv_{G_2}^{AKE}(\mathcal{A}) - Adv_{G_1}^{AKE}(\mathcal{A})| \leq (q_{ex} + q_{se} + q_{ro})(q_{ex} + q_{se})/q^{nk}$.

G_3 : When the adversary $\mathcal{A}$ does not use the random oracle model in the output of *Execute* and *Send* queries, G_3 is same as G_2 . The oracle queries by the adversary will be as consistent as possible with *Execute* and *Send* queries.

In the *Execute*($\Pi_U^x, \Pi_{Si}^x, \Pi_{Sk}^x$) query, $b_U = As_U + e_U, b_{Si} = A^T s_{Si} + e_{Si}, b_{Sk} = A^T s_{Sk} + e_{Sk}$, where $s_U, e_U, s_{Si}, e_{Si}, s_{Sk}, e_{Sk}$ is taken randomly from the distribution β_{η}^k , $\text{Auth}_{U,Sk}, \text{Auth}_{Si,Sk}$ random uniform is taken from the distribution $\{0, 1\}^d$, and c_k, c_1 is the generated uniformly at random from distribution $\{0, 1\}$. If the query

occurs between the user Π_U^x and Π_{Si}^x , it may makes the protocol to terminate, resulting in the adversary $\mathcal{A}$ failing.

Lemma 5: For any polynomial adversary, $|Adv_{G_3}^{Ake}(\mathcal{A}) - Adv_{G_2}^{Ake}(\mathcal{A})| \leq q_{se}/2^d + q_{ro}/q^{nk}$.

Proof: In G_3 , without using random oracle queries, the adversary $\mathcal{A}$ needs to provide the correct signal value as the input for $H_1(\cdot)$ in order to output the correct $Auth_{U,Sk}$ and $Auth_{Si,Sk}$. Since it is assumed that all servers except the gateway server can be compromised by the adversary, and the adversary can't obtain the correct coordination value w'_{si} , then they will not be able to compute the correct signal value c'_{i2} . Based on the analysis, the probability of obtaining the correct authentication information does not exceed q_{ro}/q^{nk} .

For the send query, the query can be divided into $Send_0(\Pi_U^x, m), Send_1(\Pi_{Si}^x, M_0), Send_2(\Pi_{Sk}^x, M_1), Send_3(\Pi_{Si}^x, M_2)$ and $Send_4(\Pi_U^x, M_3)$. In these queries, it may lead to failure in the user or server instance query, causing the protocol to terminate. In the server S_i instance, it is necessary to verify $Auth'_{Si,Sk}$ and $Auth_{Si,Sk}$. If $Auth'_{Si,Sk} \neq Auth_{Si,Sk}$, the protocol terminates; if $Auth'_{Si,Sk} = Auth_{Si,Sk}$, the protocol continues. In the user instance, we need to verify $Auth'_{Si,U}$ and $Auth_{Si,U}$. If $Auth'_{Si,U} \neq Auth_{Si,U}$, the protocol terminates; if $Auth'_{Si,U} = Auth_{Si,U}$, the protocol continues. The probability of termination is $q_{se}/2^d$.

For any polynomial adversary, the advantage of distinguishing G_3 and G_2 is $|Adv_{G_3}^{Ake}(\mathcal{A}) - Adv_{G_2}^{Ake}(\mathcal{A})| \leq q_{se}/2^d + q_{ro}/q^{nk}$.

G_4 : G_4 is that in $H_1(TID_{Uk}||n_2||n_{k4}||c'_k||b_U||b_{Sk}||PW_{Sk})$ query executes a random response without checking the consistency of its output with the execute query.

Lemma 6: For any polynomial adversary, the advantage of distinguishing G_4 and G_3 is $|Adv_{G_4}^{Ake}(\mathcal{A}) - Adv_{G_3}^{Ake}(\mathcal{A})| \leq 2Adv_{R_q}^{PWE}(t', q_{ro}) + 2Adv_{R_q}^{DMLWE}(t', q_{ro})$.

Proof: If the adversary $\mathcal{A}$ successfully implements mutual authentication between the user and the server, and then continues to perform the query $H_1(TID_{Uk}||n_2||n_{k4}||c'_k||b_U||b_{Sk}||PW_{Sk})$ without checking the consistency of the query results. It is that the adversary $\mathcal{A}$ needs to verify the consistency of values such as b_U . Then simulator can construct an algorithm D to solve MPWE problem by running the adversary on G_3 . Given (A, X, Y, σ) , the algorithm simulates the game G_3 by changing it as follows. (1) when adversary makes Execute query, the algorithm D sets $b_U = X + A^T s_U + e_U, b_{Sk} = Y + A^T s_{Sk} + e_{Sk}$, where s_U, e_U, s_{Sk}, e_{Sk} are taken from the distribution β_{η}^k . At the same time, it is assumed that the adversary knows and selects from randomly and uniformly. This assumption will increase the advantage of the adversary's successful attack. (2) When the adversary finishes, for the $H_1(TID_{Uk}||n_2||n_{k4}||c'_k||b_U||b_{Sk}||PW_{Sk})$ query, (b_U, b_{Sk}, n_2, n_3) is obtained in the execute query. Then the algorithm D can compute:

$$w_k = b_U^T s_{Sk} + e_{Sk} = (X + A^T s_U + e_U)^T (s_y + s_{Sk}) + e_{Sk} = X^T s_y + X^T s_{Sk} + s_U^T A^T s_y + s_U^T A^T s_{Sk} + e_U^T s_y + e_U^T s_{Sk} + e_{Sk} \approx X^T s_y + s_U^T Y + (X + A s_U + e_U)^T s_{Sk} \quad (4)$$

Calculate $c'_k = rec(2w_k - s_{Uk}^T Y - (X + A s_{Uk} + e_{Uk})^T s_{Sk}, \sigma_k)$ and add c'_k to the list of possible values of the MPWE problem.

when the algorithm D simulates the game G_3 , algorithm D sets $b_U = X + A^T s_U + e_U, b_{Sk} = Y + A^T s_{Sk} + e_{Sk}$ to replace actual b_U, b_{Sk} respectively. Considering the difficulty of the MPWE problem, assuming t' is the distinguisher D 's running time, and $t' = O(t + (q_{se} + q_{ex} + q_{re} + q_{co})t_{exp})$. For polynomial adversaries $\mathcal{A}$, the advantage of distinguishing between games G_4 and G_3 games is $|Adv_{G_4}^{Ake}(\mathcal{A}) - Adv_{G_3}^{Ake}(\mathcal{A})| \leq 2Adv_{R_q}^{PWE}(t', q_{ro}) + 2Adv_{R_q}^{DMLWE}(t', q_{ro})$.

G_5 : In this game, assuming that $N-1$ servers, including the gateway server, have been compromised, the password information stored on these servers is also compromised by the adversary $\mathcal{A}$. If the adversary $\mathcal{A}$ attempts a password-guessing attack on the matching user instance Π_U^x and the server Π_{Sk}^y instance, the protocol terminates.

Lemma 7: For any polynomial adversary, the advantage of distinguishing G_5 and G_4 is $|Adv_{G_5}^{Ake}(\mathcal{A}) - Adv_{G_4}^{Ake}(\mathcal{A})| \leq 2Adv_{R_q}^{PWE}(t', q_{ro}) + 2Adv_{R_q}^{DMLWE}(t', q_{ro})$.

Proof: In this game, the event *pairedpwguess* is defined as the adversary $\mathcal{A}$ performing a password-guessing attack on a pair of related user and server instances. If this event does not occur, G_5 and G_4 are indistinguishable. Given (A, X, Y, σ) , the algorithm D simulates the game G_4 by changing followings. In $Send_0(\Pi_U^x, m)$ query to user instance Π_U^x , the algorithm D sets $b_U = X$. In $Send_2(\Pi_{Sk}^y, M_1)$ query to server instance Π_{Sk}^y . Set $b_{Sk} = Y + A^T s_{Sk} + e_{Sk}$, where s_{Sk}, e_{Sk} are all taken from β_{η}^k . In $Send_4(\Pi_U^x, M_3)$ query to Π_U^x , if Π_U^x has not been paired, the algorithm outputs 0 and aborts.

After the adversary attack finished, for each $H_1(TID_{Uk}||n_2||n_{k4}||c'_{i2}||b_U||b_{Si}||PW_{Si})$ query, when b_U and b_{Sk} are in a related Π_U^x query, the algorithm D can calculate as follows:

$$w_k = b_U^T s_{Sk} + e_{Sk} = X^T (s_y + s_{Sk}) + e_{Sk} = X^T s_y + X^T s_{Sk} + e_{Sk} \approx X^T s_y + X^T s_{Sk} \quad (5)$$

Calculate $c'_k = rec(2w_k - s_{Uk}^T Y - (X + A s_{Uk} + e_{Uk})^T s_{Sk}, \sigma_k)$, and add c'_k to the list of possible values of the MPWE problem. when the algorithm D simulates the game G_4 , algorithm D sets $b_U = X, b_{Sk} = Y + A^T s_{Sk} + e_{Sk}$ to replace actual $b_U, b_{Sk} = A^T s_{Sk} + e_{Sk}$. Considering the difficulty of the MPWE problem, assuming t' is the distinguisher D 's running time, and $t' = O(t + (q_{se} + q_{ex} + q_{re} + q_{co})t_{exp})$. For adversaries $\mathcal{A}$, the advantage of distinguishing between games G_5 and G_4 games is $|Adv_{G_5}^{Ake}(\mathcal{A}) - Adv_{G_4}^{Ake}(\mathcal{A})| \leq 2Adv_{R_q}^{PWE}(t', q_{ro}) + 2Adv_{R_q}^{DMLWE}(t', q_{ro})$.

G_6 : G_6 is that there is an internal password oracle that keeps all passwords and is used to check the correctness of a given password in G_6 . The password oracle initializes passwords and is unavailable to arbitrary polynomial adversaries. During the initialization process, this oracle generates all user passwords, but the adversary cannot use this password oracle.

Lemma 8: For any polynomial adversary, G_6 and G_5 are indistinguishable: $Adv_{G_6}^{Ake}(\mathcal{A}) = Adv_{G_5}^{Ake}(\mathcal{A})$.

Proof: In this section, we analyze the advantages of the adversarial attack game G_6 . According to the definition of the game G_6 , this section can easily bound the probability of the adversary's success in the game as follows.

$$Pr[Succ_{G_6}] \leq Pr[Correctpw] + Pr[Succ_{G_6} | \neg Correctpw] \cdot Pr[\neg Correctpw] \quad (6)$$

In G_6 , the adversary $\mathcal{A}$ has two ways to succeed: one is by performing an online guessing attack, and the other is by making Test query. Define event *Correctpw* as the event where adversary $\mathcal{A}$ successfully uses an online guessing attack to obtain the password for a fresh session. When the adversary chooses a password from a dictionary of length L and can make at most $Send$ queries, we can calculate that the probability of success is $Pr[Correctpw] \leq q_{se}/|L|$.

For a fresh session instance, if event *Correctpw* does not occur, the adversary can infer the correct session key based on the related information it has obtained. Therefore, the adversary can only succeed if it performs Test query and successfully guesses b . The probability of success is $1/2$.

$$Pr[Succ_{G_6} | \neg Correctpw] \cdot Pr[\neg Correctpw] \leq 1/2(1 - q_{se}/|L|) \quad (7)$$

$$Pr[Succ_{G_6}] \leq q_{se}/L + 1/2(1 - q_{se}/|L|) \leq 1/2 + q_{se}/2|L| \quad (8)$$

Integrating G_0 to G_6 , since the proof assumes that the gateway server is secure, we can also assume that all other $N-1$ servers may not be compromised. Based on the analysis, the probability of the adversary's successful attack is $Adv_{\Pi}^{Ake}(\mathcal{A}) \leq (q_{ro} + q_{se} + q_{ex})(q_{ex} + q_{se})/q^{nk} + q_{se}/2^d + q_{ro}/q^{nk} + 4Adv_{R_q}^{PWE}(t', q_{ro}) + 4Adv_{R_q}^{DMLWE}(t', q_{ro}) + q_{se}/|L|$ and its value is negligible. The attacker bases the query on the random oracle, and the advantage of a successful attack is almost 0.

5.2 Protocol 2

In this protocol, a separate session key is generated between multiple servers and users, resembling the structure of a single server.

5.2.1 System initialization process. (1) $U \rightarrow S_k: (ID_U, A, PW_{S_k})$.

When the user U joins the protocol system, the user U sends registration information to the servers S_k ($2 \leq k \leq N$). The identity information ID_U and password PW_U are chosen randomly for each user U and we assume that password can be mapped in a one-to-one fashion to Z_q . The user U randomly selects $A \leftarrow R_q^{k \times k}$. If S_k are the servers associated with the user U , then shares $PW_{S_k} \in Z_q$ are chosen uniformly at random subject to $\sum_{k=1}^N PW_U^k \pmod{q} = PW_U$, PW_{S_k} given to server S_k .

(2) $S_k \rightarrow U: (TID_{Uk})$.

After receiving U 's registration information, S_k adds $\langle ID_U, PW_{S_k} \rangle$ to its database. Then we use a random number n_1 and the identity information ID_U to compute a temporary identity $TID_{Uk} = H_0(ID_U || n_1)$ for user U . And Server S_k saves TID_{Uk} and sends TID_{Uk} to U .

5.2.2 Authentication and key exchange phase. When user U needs to share the session key for communication, it is essential to establish authentication between users U with the server S_k . This ensures the legitimacy of identities and facilitates the exchange of

shared session keys. The authentication and key exchange process is shown in Figure 2.

1. $U \rightarrow S_k: M_0 = (TID_{Uk}, n_{k2}, m_{Uk})$.

The user U samples $(s_{Uk}, e_{Uk}) \leftarrow \beta_{\eta}^k \times \beta_{\eta}^k$ uniformly, generates random number n_{k2} , and calculates $b_{Uk} = As_{Uk} + e_{Uk}$, $\gamma_0 = H_1(TID_{Uk} || n_{k2} || PW_{S_k})$ and $m_{Uk} = \gamma_0 + b_{Uk}$. The user U sends $M_0 = (TID_{Uk}, n_{k2}, b_{Uk})$ to the server S_k .

2. $S_k \rightarrow U: M_1 = (Auth_k, n_{k3}, m_{S_k})$.

Upon receiving the request, the server confirms the identity of the user applying for authentication and key exchange through TID_{Uk} . Then the server S_k samples $(s_{Sk}, e_{Sk}) \leftarrow \beta_{\eta}^k \times \beta_{\eta}^k$ and $(e_{Sk3}, \tilde{e}_k) \leftarrow \beta_n \times \beta_n$ uniformly, generates random number n_{k3} and calculates $b_{Sk} = A^T s_{Sk} + e_{Sk} \gamma_0$ and $w_k = b_{Uk}^T s_{Sk} + e_{Sk1}$. Next, we need calculate $b_{Uk} = m_{Uk} - \gamma_0$, $\gamma_1 = H_1(TID_{Uk} || n_{k2} || n_{k3} || PW_{S_k})$, $m_{S_k} = \gamma_1 + b_{S_k}$. Next, the server S_k uses the randomized function, cross-rounding function and modular rounding function to calculate $\tilde{w}_k \leftarrow dbl(w_k)$, $\sigma_k \leftarrow \langle \tilde{w}_k \rangle_{2q,2}$, $c_k \leftarrow \lfloor \tilde{w}_k \rfloor_{2q,2}$. At the meanwhile, the server S_k calculates authentication information $Auth_k = H_1(TID_{Uk} || n_{k2} || n_{k3} || c_k || b_{S_k} || PW_{S_k})$ between user and server S_k . The server S_k calculates the shared key $SK_{U,k} = H_2(TID_{Uk} || n_{k2} || n_{k3} || c_k || b_{Uk} || b_{S_k})$ between U and S_k . The server S_k sends M_1 to the server S_k .

3. $U: SK$.

After the user receives the message M_1 , the user calculates $\gamma_1 = H_1(TID_{Uk} || n_{k2} || n_{k3} || PW_{S_k})$ and $b_{S_k} = m_{S_k} - \gamma_1$, $w_k' = b_{S_k}^T s_{Uk}$. Then it uses Peikert error reconciliation function to calculate $c_k' = rec(2w_k', \sigma_k)$. Next, the user calculates the verification information $Auth_k' = H_1(TID_{Uk} || n_{k2} || n_{k3} || c_k' || b_{S_k} || PW_{S_k})$ with the server S_k . If $Auth_k' = Auth_k$, the user continues to calculate the shared key $SK_{U,k} = H_2(TID_{Uk} || n_{k2} || n_{k3} || c_k || b_{Uk} || b_{S_k})$ with the server S_k .

5.2.3 Security analysis. This section provides a security proof of the PAKE protocol proposed in this paper based on the BPR model. In this scheme, a single distinct session key is generated between multiple servers and user, and we can just consider only the security between a single server and user. Based on the security model defined in Section 4, we have the following theorem:

Theorem 4: Let Π be the protocol proposed in this paper, and L be the dictionary library of size $|L|$. $q_{ex}, q_{se}, q_{re}, q_{co}$ respectively represent Execute, Send, Reveal, Corrupt queries, and q_{ro} represent querying random oracles. Assume that the adversary $\mathcal{A}$ can control the simulator access to all oracles that. The simulator runs the protocol Π . For the adversary $\mathcal{A}$, the advantage of its attack protocol is defined:

$$Adv_{\Pi}^{AKE}(\mathcal{A}) \leq (q_{ex} + q_{se} + q_{ro}) \left((q_{ex} + q_{se})/q^{nk} + q_{ro}/q^{nk} + q_{se}/2^d \right) + 4Adv_{R_q}^{PWE}(t, q_{ro}) + q_{se}/L \quad (9)$$

Proof. The game G_i ($i = 0, \dots, 6$) is defined in the security model of this section. $G_0 = \Pi$ corresponds to the honest implementation of the protocol. G_6 involves simulating the protocol in an ideal situation under the random oracle model, during which the adversary's advantage in attacking the protocol is negligible. The advantage of the adversary attacking the protocol in G_i is greater than that of the adversary attacking the protocol in G_{i-1} , the security of the protocol is gradually reduced, we can conclude the

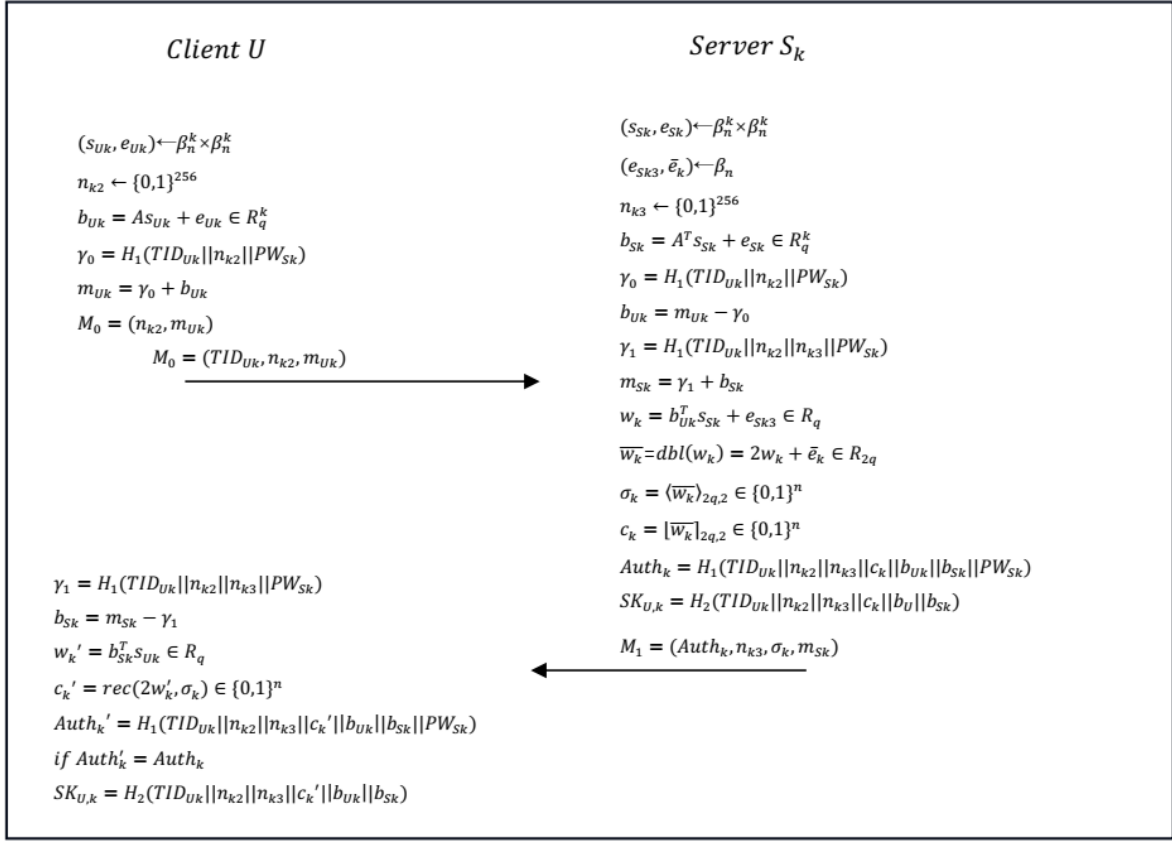


Figure 2: Authentication and key exchange phase

following:

$$\begin{aligned} Adv_{G_0}^{Ake}(\mathcal{A}) &\leq Adv_{G_1}^{Ake}(\mathcal{A}) + \varepsilon_1 \\ &\leq Adv_{G_2}^{Ake}(\mathcal{A}) + \varepsilon_2 \leq Adv_{G_3}^{Ake}(\mathcal{A}) + \varepsilon_3 \leq Adv_{G_4}^{Ake}(\mathcal{A}) \quad (10) \\ +\varepsilon_4 &\leq Adv_{G_5}^{Ake}(\mathcal{A}) + \varepsilon_5 \leq Adv_{G_6}^{Ake}(\mathcal{A}) + \varepsilon_6 \end{aligned}$$

For games G_0, G_1, G_2 the proof process is consistent with Protocol 1. We will prove it from the beginning of the game G_3 .

G_3 : When the adversary $\mathcal{A}$ does not use the random oracle model in the output of Execute and Send queries, G_3 is same as G_2 . The oracle queries by the adversary will be as consistent as possible with Execute and Send queries. In the Execute query, $b_{Uk} = A s_{Uk} + e_{Uk}$, $b_{Sk} = A^T s_{Sk} + e_{Sk}$, where $s_{Uk}, e_{Uk}, s_{Sk}, e_{Sk}, e_{Sk3}, \bar{e}_k$ is taken randomly from the distribution β_n^k , $Auth_k$ random uniform is taken from the distribution $\{0,1\}^d$. If the query occurs between the user Π_U^x and Π_{Sk}^x , it may makes the protocol to terminate, resulting in the adversary $\mathcal{A}$ failing.

Lemma 9: For any polynomial adversary, $|Adv_{G_3}^{Ake}(\mathcal{A}) - Adv_{G_2}^{Ake}(\mathcal{A})| \leq q_{se}/2^d + q_{ro}/q^{nk}$.

Proof: In G_3 , without using random oracle queries, the adversary $\mathcal{A}$ provides the correct signal value as the input for $H_1(\cdot)$ in order to output the correct $Auth_k$. The adversary can't obtain the correct coordination value w_k , then they will not be able to compute the correct signal value c_k' . Based on the analysis,

the probability of obtaining the correct authentication information does not exceed q_{ro}/q^{nk} . It may lead to failure in the user or server instance query, causing the protocol to terminate. In the user U instance, it is necessary to verify $Auth_k'$ and $Auth_k$. If $Auth_k' \neq Auth_k$, the protocol terminates. The probability of termination is $q_{se}/2^d$. For the adversary $\mathcal{A}$, the advantage of distinguishing G_3 and G_2 is $|Adv_{G_3}^{Ake}(\mathcal{A}) - Adv_{G_2}^{Ake}(\mathcal{A})| \leq q_{se}/2^d + q_{ro}/q^{nk}$.

G_4 : G_4 is that in $H_1(TID_{Uk} || n_{k2} || n_{k3} || c_k' || b_{Uk} || b_{Sk} || PW_{Sk})$ query executes a random response without checking the consistency of its output with the Execute query.

Lemma 10: For any polynomial adversary, the advantage of distinguishing G_4 and G_3 is $|Adv_{G_4}^{Ake}(\mathcal{A}) - Adv_{G_3}^{Ake}(\mathcal{A})| \leq 2Adv_{R_q}^{PWE}(t', q_{ro})$.

Proof: If the adversary $\mathcal{A}$ successfully implements mutual authentication between the user and the server, and then continues to perform the query $H_1(TID_{Uk} || n_{k2} || n_{k3} || c_k' || b_{Uk} || b_{Sk} || PW_{Sk})$ without checking the consistency of the query results. It is that the adversary $\mathcal{A}$ needs to verify the consistency of values such as b_{Uk} . Given (A, X, Y, σ) , the algorithm simulates the game G_3 by changing it as follows.

(1) When adversary makes Execute query, the algorithm D sets $b_{Uk} = X + A^T s_{Uk} + e_{Uk}$, $b_{Sk} = Y + A^T s_{Sk} + e_{Sk}$, where

$s_{Uk}, e_{Uk}, s_{Sk}, e_{Sk}$ are taken from the distribution β_{η}^k . At the same time, it is assumed that the adversary knows and selects from randomly and uniformly. This assumption will increase the advantage of the adversary's successful attack.

(2) When the adversary finishes, for the $H_1(TID_{Uk}||n_{k2}||n_{k3}||c'_k||b_{Uk}||b_{Sk}||PW_{Sk})$ query, $b_{Uk}, b_{Sk}, n_{k2}, n_{k3}$ is obtained in the execute query. And we can look up $\gamma_0 = As_{h0} + e_{h0}$, $\gamma_1 = As_{h1} + e_{h1}$ in a randomized prediction machine, and Algorithm D can compute. Then the algorithm D can compute:

$$\begin{aligned} w_k &= b_{Uk}^T s_{Sk} + \bar{e}_{Sk} = (X + A^T s_{Uk} + e_{Uk} - As_{h0} - e_{h0})^T \\ &\quad (s_y + s_{Sk} - s_{h1}) + \bar{e}_{Sk} = X^T s_y + X^T (s_{Sk} - s_{h1}) \\ &\quad + (s_{Uk} - s_{h0})^T A^T (s_{Sk} - s_{h1}) + (s_{Uk} - s_{h0})^T A^T s_y + (e_{Uk} - e_{h0})^T s_y \\ &\quad + (e_{Uk} - e_{h0})^T (s_{Sk} - s_{h1}) + \bar{e}_{Sk} \approx X^T s_y + (s_{Uk} - s_{h0})^T Y \\ &\quad + (X + As_{Uk} + e_{Uk} - As_{h0} - e_{h0})^T (s_{Sk} - s_{h1}) \end{aligned} \quad (11)$$

Calculate $c'_k = \text{rec}(2(w_k - (s_{Uk} - s_{h0})^T Y - (X + As_{Uk} + e_{Uk} - As_{h0} - e_{h0})^T (s_{Sk} - s_{h1})), c_k)$ and add c'_k to the list of possible values of the MPWE problem. when the algorithm D simulates the game G_3 , algorithm D sets $b_{Uk} = X + A^T s_{Uk} + e_{Uk}$, $b_{Sk} = Y + A^T s_{Sk} + e_{Sk}$ to replace actual b_{Uk}, b_{Sk} respectively. Considering the difficulty of the MPWE problem, assuming t' is the distinguisher D's running time, and $t' = O(t + (q_{se} + q_{ex} + q_{re} + q_{co})t_{exp})$. For polynomial adversaries $\mathcal{A}$, the advantage of distinguishing between games G_4 and G_3 games is $|Adv_{G_4}^{Ake}(\mathcal{A}) - Adv_{G_3}^{Ake}(\mathcal{A})| \leq 2Adv_{Rq}^{PWE}(t', q_{ro})$.

G_5 : If the adversary $\mathcal{A}$ attempts a password-guessing attack on the matching user instance Π_U^x and the server Π_{Sk}^y instance, the protocol terminates and the adversary is considered to have failed.

Lemma 11: For any polynomial adversary, the advantage of distinguishing G_5 and G_4 is $|Adv_{G_6}^{Ake}(\mathcal{A}) - Adv_{G_5}^{Ake}(\mathcal{A})| \leq 2Adv_{Rq}^{PWE}(t', q_{ro})$.

Proof: In this game, the event *pairedpwguess* is defined as the adversary $\mathcal{A}$ performing a password-guessing attack on a pair of related user and server instances. If this event does not occur, then G_5 and G_4 are indistinguishable. Given (A, X, Y, σ) , the algorithm D simulates the game G_4 by changing followings.

In $Send_0(\Pi_U^x, m)$ query to user instance Π_U^x , the algorithm D sets $b_U = X$. In $Send_2(\Pi_{Sk}^y, M_1)$ query to server instance Π_{Sk}^y . Set $b_{Sk} = Y + A^T s_{Sk} + e_{Sk}$, where s_{Sk}, e_{Sk} are all taken from β_{η}^k . In $Send_4(\Pi_U^x, M_3)$ query to user instance Π_U^x , if Π_U^x has not been paired, the algorithm outputs 0 and aborts.

After the adversary attack finished, for each $H_1(TID_{Uk}||n_{k2}||n_{k3}||c'_k||b_{Uk}||b_{Sk}||PW_{Sk})$ query, when b_U and b_{Sk} are in a related Π_U^x query, the algorithm D can calculate as follows:

$$\begin{aligned} w_{Sk} &= b_U^T s_{Sk} + e_{Sk} = (X - As_{h0} - e_{h0})^T (s_y + s_{Sk} - s_{h1}) + e_{Sk} \\ &= X^T s_y + X^T (s_{Sk} - s_{h1}) - s_{h0}^T A^T \\ &\quad (s_{Sk} - s_{h1}) - e_{h0}^T s_y - e_{h0}^T (s_{Sk} - s_{h1}) + e_{Sk} \\ &\approx X^T s_y + s_{h0}^T Y + (X - As_{h0} - e_{h0})^T (s_{Sk} - s_{h1}) \end{aligned} \quad (12)$$

Calculate $c'_k = \text{rec}(2(w_k - (s_{Uk} - s_{h0})^T Y - (X + As_{Uk} + e_{Uk} - As_{h0} - e_{h0})^T (s_{Sk} - s_{h1})), c_k)$, and add

c'_k to the list of possible values of the MPWE problem. when the algorithm D simulates the game G_5 , algorithm D sets $b_U = X$, $b_{Sk} = Y + A^T s_{Sk} + e_{Sk}$ to replace actual $b_U, b_{Sk} = A^T s_{Sk} + e_{Sk}$. Considering the difficulty of the MPWE problem, assuming t' is the distinguisher D's running time, and $t' = O(t + (q_{se} + q_{ex} + q_{re} + q_{co})t_{exp})$. For probabilistic polynomial adversaries $\mathcal{A}$, the advantage of distinguishing between games G_5 and G_4 games is $|Adv_{G_5}^{Ake}(\mathcal{A}) - Adv_{G_4}^{Ake}(\mathcal{A})| \leq 2Adv_{Rq}^{PWE}(t', q_{ro})$.

G_6 : G_6 is that there is an internal password oracle that keeps all passwords and is used to check the correctness of a given password in G_6 . The password oracle initializes passwords and is unavailable to arbitrary polynomial adversaries. During the initialization process, this oracle generates all user passwords, but the adversary cannot use this password oracle.

Lemma 12: For any polynomial adversary, G_6 and G_5 are indistinguishable: $Adv_{G_6}^{Ake}(\mathcal{A}) = Adv_{G_5}^{Ake}(\mathcal{A})$.

Proof: In this section, we analyze the advantages of the adversarial attack game G_6 . According to the definition of the game G_6 , this section can easily bound the probability of the adversary's success in the game as follows.

$$\begin{aligned} Pr[Succ_{G_6}] &\leq Pr[Correctpw] \\ &+ Pr[Succ_{G_6} | \neg Correctpw] \cdot Pr[\neg Correctpw] \end{aligned} \quad (13)$$

In G_6 , the adversary $\mathcal{A}$ has two ways to succeed: one is by performing an online guessing attack, and the other is by making Test query. Define event *Correctpw* as the event where adversary $\mathcal{A}$ successfully uses an online guessing attack to obtain the password for a fresh session. When the adversary chooses a password and can make at most *Send* queries, we can calculate that the probability of success is $Pr[Correctpw] \leq q_{se}/|L|$. For a fresh session instance, if event *Correctpw* does not occur, the adversary can infer the correct session key based on the related information. The adversary can only succeed if it performs Test query and successfully guesses b . The probability of success is 1/2.

$$Pr[Succ_{G_6} | \neg Correctpw] \cdot Pr[\neg Correctpw] \leq 1/2(1 - q_{se}/|L|) \quad (14)$$

$$Pr[Succ_{G_6}] \leq q_{se}/L + 1/2(1 - q_{se}/|L|) \leq 1/2 + q_{se}/2|L| \quad (15)$$

Integrating G_0 to G_6 , since the proof assumes that the gateway server is secure, we can also assume that all other N-1 servers may not be compromised. Based on the above analysis, the probability of the adversary's successful attack is $Adv_{\Pi}^{Ake}(\mathcal{A}) \leq (q_{ro} + q_{se} + q_{ex})(q_{ex} + q_{se})/q^{nk} + q_{se}/2^d + q_{ro}/q^{nk} + 4Adv_{Rq}^{PWE}(t', q_{ro}) + q_{se}/|L|$ and its value is negligible. The attacker bases the query on the random oracle, and the advantage of a successful attack is almost 0.

6 COMPARISON OF PROTOCOL PERFORMANCE

6.1 Computational Efficiency

All experimental results were encoded using C language, the software implementation of the protocol presented in this paper was executed on a 64-bit system computer with 2.90GHz 11th Gen Intel(R) Core (TM) i7-10700 and 16GB RAM on Windows 11 Version 22H2. To ensure the correctness and post-quantum security of the

Table 2: Average running time of various calculations (in microsecond)

Operation	T_A	T_S	T_e
time	28.8	6.2	6.0
Operation	T_{add}	T_{mul}	T_{muladd}
time	3.0	13.2	16.5
Operation	T_{NTT}	T_{iNTT}	T_k
time	27.2	41.5	155.3

protocol, the protocol is based on the MLWE problem. The scheme chooses parameters such as the vector dimension $n = 256$, modulus $q = 3329$, polynomial matrix dimension $k = 3$, and the center binomial distribution parameter $\eta = 2$. To improve the computational efficiency of the protocol, the NTT algorithm is used to optimize the protocol, reducing the complexity of polynomial operations.

6.2 Computational Efficiency Algorithm Execution Time

In this section, several types of operations with relatively high computational complexity are selected for experimentation, including central binomial distribution sampling, polynomial operations, and key generation operations. At the same time, based on the MLWE parameters provided in Section 6.1, the above operations are executed 10,000 times, and the average value is taken as the experimental result. Table 2 presents the average execution time for each operation in the protocol.

- T_A is the time to generate the public parameters. T_S , T_e is the time for sampling random error values from the central binomial distribution. T_{add} is the time to add two ring elements. T_{mul} is the time to multiply two ring elements. T_{muladd} is the time to multiply two ring elements and then add them. T_{NTT} is the time to convert a ring element to an NTT domain element. T_{iNTT} is the time to convert an NTT domain element back to a ring element. T_k is the time for key generation operations.

6.3 Experimental Results Analysis and Comparison

To facilitate the comparison of the protocol's security, this paper is compared with the literature [25, 27, 35, 42] in security, as shown in Table 3. In terms of protocol types, the PAKE protocols from [35, 42] all utilize single-service architectures and achieve three-party mutual authentication. The literature referenced as [25, 27] describes a two-server architecture that effectively resists server leakage attacks. However, Yi's protocol presented in [25], do not utilize password-only setups.

In contrast, the two schemes discussed in this paper, as well as the ones in [35, 42], all implement password-only setups. For the security, the selected protocols all implement forward security, while the literature refers to traditional mathematically difficult problems, such as large integer factorization and discrete logarithm problems, which are not resistant to known quantum attacks. Both the literature and the protocol discussed in this paper are based on the Lattice Difficulty Problem, which offers resistance to known

quantum attacks. The literature by Shu's protocol [35], adopts a verification meta-technique that can mitigate the risk of server leakage attacks. In contrast, the two schemes successfully address server leakage attacks and employ the random predicate machine model for security proof, demonstrating that the PAKE protocol has higher security.

The literature [25, 27] describes a protocol designed around traditional mathematical hard problems, employing numerous costly cryptographic primitives such as signatures and secret sharing. This results in significant increases in both computational and communication overhead, making it difficult to draw comparisons regarding these overheads. In contrast, this paper presents a protocol based on lattice hard problems. We focus on the same lattice hard problem used in the protocol for a comparative analysis of efficiency and performance [35, 42]. The parameters of our protocol are established, specifically setting the modulus related to the RLWE problem and the public key length to 256 bits. The performance outcomes are compared and summarized in Tables 4

In terms of communication overhead, experimental comparisons and analyses indicate that the two schemes proposed in this paper reduce the number of communications involving the server, compared to other protocol [35, 42]. This reduction aligns with the findings in the literature, demonstrating lower communication overhead. Additionally, the proposed protocol enhances efficiency by minimizing the communication burden between the user and the server.

In order to maintain a relatively similar level of security between Shu's, Guo's protocols and our Scheme, the experiment set parameters for both the RLWE problem. Specifically, the parameters for the RLWE problem included a modulus $q = 3329$, the vector dimension $n = 768$, and a public key length of 256 bits. These configurations are used to simulate the protocol execution and measure the protocol's execution time, which reflects its computational overhead. We can gain $T_{ARLWE} = 26.5$, $T_{sRLWE} = 10.6$, $T_{eRLWE} = 10.4$ (T_{ARLWE} is the time to generate the public parameters. T_{sRLWE} , T_{eRLWE} is the time for sampling random error values from the central binomial distribution).

To more effectively improve computational efficiency, we utilize the more computationally efficient Peikert error-coordination mechanism. This approach effectively reduces computational overhead, enhances usability, and minimizes computational costs. At the meanwhile, in order to improve the efficiency of sampling while ensuring indistinguishability between sampling results and discrete Gaussian distribution sampling, we use centered binomial distribution sampling.

Table 3: Safety comparison

Protocol	Shu Scheme [35]	Guo Scheme [42]	Yi Scheme [25]	Kumari Scheme [27]	protocol 1	protocol 2
Protocol type	Single server	Single server	Two-server	Two-server	Multi-server	Multi-server
Difficult hypothesis	RLWE	MLWE	DDH	ECDH	MLWE	MLWE
Password-only	✓	✓	×	✓	✓	✓
Communication rounds	9	4	6	6	2N+2	2N
Quantum-resistance	✓	✓	×	×	✓	✓
Server leakage attack	×	×	✓	✓	✓	✓
Undetectable online dictionary attack	×	✓	×	×	✓	✓
Security model	Standard model	Standard model	Standard model	Standard model	BPR model	BPR model

Table 4: Comparison of computing costs

Protocol	Computing cost/ms
Shu Scheme [35]	$T_{ARLWE} + 6T_{sRLWE} + 10T_{eRLWE} + 12T_{NTT} + 4T_{add} + 12T_{muladd}$
Guo Scheme [42]	$T_A + 6T_s + 7T_e + 9T_{NTT} + 9T_{iNTT} + 3T_{mul} + 6T_{muladd}$
Scheme 1	$T_A + (N + 2)T_s + (5N + 4)T_e + (7N + 2)T_{NTT} + (7N + 2)T_{iNTT} + 3NT_{mul} + (4N + 2)T_{muladd} + NT_{add}$
Scheme 2	$T_A + 2NT_s + 4NT_e + 4NT_{NTT} + 4NT_{iNTT} + 5NT_{add} + 3NT_{muladd}$

7 CONCLUSION

In this paper, we propose two multi-server password authentication key exchange protocols based on lattice:

1. Protocol 1 is primarily suited for high-security application scenarios. In this scheme, a single shared session key is generated between the user and all servers upon completion of the protocol.

2. Protocol 2 is designed for high-efficiency applications. The user generates an independent session key with each server, enhancing communication efficiency and reducing computational overhead.

Both schemes are founded on the MLWE problem, which offers better computational performance and higher security compared to the LWE and RLWE problems, thereby effectively resisting known quantum attacks. Additionally, we employ the Peikert error coordination mechanism, which is more computationally efficient. This mechanism reduces the number of interaction rounds required in the protocols, which effectively lowers communication overhead and protects against server leakage attacks.

Acknowledgments

This project was supported by the Information Engineering University.

References

- [1] Ji Sun Shin, Minjae Jo, Jung Yeon Hwang, Jaehwan Lee. **2021**. A verifier-based password-authenticated key exchange using tamper-proof hardware. *The Computer Journal*, 64(8), 1293-1302.
- [2] Shor, Peter W. **1999**. Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer. *SIAM review*, 41(2), 303-332.
- [3] Grover Lov K. **1996**. A fast quantum mechanical algorithm for database search. *Proceedings of the twenty-eighth annual ACM symposium on Theory of computing*, 212-219.
- [4] Asif R. **2021**. Post-quantum cryptosystems for IOT: a survey on lattice-based algorithms. *IoT*, 2(1), 71-91.
- [5] Roy Partha Sarathi, Dutta Sabyasachi, Susilo Willy, Safavi-Naini Reihaneh. **2021**. Password protected secret sharing from Lattices. *International Conference on Applied Cryptography and Network Security*, 442-459.
- [6] Seyhan Kübra, Nguyen Tu N, Akleylek Sedat, Cengiz Korhan. **2022**. "Lattice-based cryptosystems for the security of resource-constrained IoT devices in post-quantum world: a survey". *Cluster Computing*, 25(3), 1729-1748.
- [7] Khan Naveed, Zhang Jian Biao, Ullah Intikhab *et al.* **2023**. Lattice-Based Authentication Scheme to Prevent Quantum Attack in Public Cloud Environment. *Computers, Materials & Continua*, 75(1), 35-49.
- [8] Alagic Gorjan, Alperin-Sheriff Jacob, Daniel Apon *et al.* **2020**. Status report on the second round of the NIST post-quantum cryptography standardization process. From <https://csrc.nist.gov/pubs/ir/8309/final>
- [9] Liu Chao, Zheng Zhong Xiang, Jia Ke Ting *et al.* **2019**. Provably Secure Three-Party Password-Based Authenticated Key Exchange from RLWE. *The 15th International Conference on Information Security Practice and Experience*, 56-72.
- [10] Gao Xin wei, Ding Jin tai, Liu Ji qiang. **2018**. Post-Quantum Secure Remote Password Protocol from RLWE Problem. *Proceedings of the 13th International Conference on Information Security and Cryptology*, 99-116.
- [11] Nguyen Khoa, Tan Benjamin Hong Meng, Wang Hua Xiong. **2017**. Zero-knowledge password policy check from lattices. *2017 International Conference on Information Security*, 92-113.
- [12] Li Zeng Peng, Wang Ding. **2019**. Two-round PAKE protocol over lattices without NIZK. *2019. Information Security and Cryptology*. 138-159.
- [13] Li Zeng Peng, Wang Ding. Achieving one-round password-based authenticated key exchange over lattices. **2022** *IEEE Transactions on Services Computing*, 15(1), 308-321.
- [14] Benhamouda Fabrice, Blazy Olivier, Ducas Léo. **2018**. Hash proof systems over lattices revisited". *IEEE Transactions on Services Computing*, 644-674.
- [15] Zhang Jiang, Yu Yu. **2017**. Two-round PAKE from approximate SPH and instantiations from lattices. *International Conference on the Theory and Application of Cryptology and Information Security*. 37-67.
- [16] Chen Lin, Qu Tong Zhou, Yin An Qi. **2024**. Quantum-safe multi-server password-based authenticated key exchange protocol. *Multimedia Tools and Applications*, 1-28.

- [17] Komal Pursharthi, Mishra Dheerendra. **2024**. Post-quantum framework for authorized and secure communication in multi-server networking. *Telecommunication Systems*, 1-16.
- [18] Kaur Sumeet, Khare Mayank Deep, Bhatt Mukul. **2024**. Multi-cloud security model: establishment of inter-server communication for authentication integrity. *International Journal of System Assurance Engineering*, 1-12.
- [19] Yin An Qi, Guo Yuan Bo, Wang Ding *et al.* **2022**. Provably secure quantum resistance two-server password-authenticated key exchange protocol. *Journal on Communications*. 43(3), 14-29.
- [20] Barman Subhas, Chattopadhyay Samiran, Barman Debasis Samanta. **2024**. A lightweight authentication protocol for a blockchain-based off-chain medical data access in multi-server environment. *SN Computer Science*, 5(3), 292.
- [21] Brainard John, Juels Ari, Kaliski Burt *et al.* **2003**. A new two-server approach for authentication with short secrets. 12th USENIX Security Symposium (USENIX Security 03), 201-213.
- [22] Katz Jonathan , MacKenzie Philip, Taban Gelareh *et al.* **2005**. Two-server password-only authenticated key exchange. *Applied Cryptography and Network Security: Third International Conference, ACNS*, 1-16.
- [23] Yang Yan Jiang, Deng Robert H, Bao Feng. **2006**. Fortifying password authentication in integrated healthcare delivery systems. *Proceedings of the 2006 ACM Symposium on Information, computer and communications security*. 255-265.
- [24] Jin Hai Min, Wong Duncan S, and Xu Yin Long. **2007**. An efficient password-only two server authenticated key exchange system. *Information and Communications Security: 9th International Conference, ICICS 2007*. 44-56.
- [25] Yi Xun., Ling San, and Wang Hua Xiong. **2013**. Efficient two-server password-only authenticated key exchange. *IEEE transactions on Parallel and Distributed systems*, 24(9), 1773-1782.
- [26] Kiefer Franziskus. **2014**. Distributed smooth projective hashing and its application to two-server password authenticated key exchange. *On Applied Cryptology and Network Security, ACNS 2014 (LNCS 8479)*, 199–216.
- [27] Kumari K Anitha. **2016**. An efficient 3d elliptic curve Diffie–Hellman (ECDH) based two-server password-only authenticated key exchange protocol with provable security. *IETE Journal of Research*, 62(6), 762-773.
- [28] Chattaraj Durbadal, Sarma Monalisa, Kumar Das Ashok. **2018**. A new two-server authentication and key agreement protocol for accessing secure cloud services. *Computer Networks*, 144–164.
- [29] Ford Warwick, Kaliski Jr Burton S. **2000**. Server-assisted generation of a strong secret from a password. *Proceedings IEEE 9th International Workshops on Enabling Technologies: Infrastructure for Collaborative Enterprises*, 176-180.
- [30] Jablon. **2001**. Password authentication using multiple servers. *Cryptographers' Track at the RSA Conference*, 344-360.
- [31] MacKenzie Philip, Shrimpton Thomas, Jakobsson Markus. **2002**. Threshold password-authenticated key exchange. *Cryptology*, 19(1), 27-66.
- [32] Katz Jonathan, Vaikuntanathan Vinod. **2009**. Smooth projective hashing and password-based authenticated key exchange from lattices. *Advances in Cryptology - ASIACRYPT 2009*. 636-652.
- [33] Yang Xiao Yan, Hou Meng Bo, Wei Xiao Chao. **2016**. Verifier-based three-party password authenticated key exchange protocol. *Journal of Computer Research and Development*, 53(10), 2230-2238.
- [34] Zhang Qi Hui, Hu Xue Xian, Liu Wen Fen *et al.* **2020**. Improved verifier-based three-party password-authenticated key exchange protocol. *Journal of Software*, 31(10), 3238-3250.
- [35] Shu Qin, Wang Sheng Bang, Hu Bin *et al.* **2021**. Verifier-based three-party password authenticated key exchange protocol. *Journal of Cryptologic Research*, 8(2), 294-306.
- [36] Langlois Adeline, Stehle Damien. **2015**. Worst-case to average-case reductions for module lattices. *Designs Codes & Cryptography*, 75(3), 565-599.
- [37] Chen Lin, Qu Tong Zhou, Yin An Qi. **2024**. Quantum-safe multi-server password-based authenticated key exchange protocol. *Multimedia Tools and Applications*, 2024, 1-28.
- [38] Bai Shi, Lepoint Tancrede, Roux-Langlois Adeline *et al.* **2015**. Improved security proofs in lattice-based cryptography: using the Rényi divergence rather than the statistical distance. *Journal of Cryptology*, 565-599.
- [39] Peikert Chris. **2014**. Lattice cryptography for the internet. *Post-quantum cryptography*, 197-219.
- [40] Ding Jin Tai, Xie Xiang, Lin Xiao Dong. **2012**. A Simple Provably Secure Key Exchange Scheme Based on the Learning with Errors Problem. *Iacr Cryptology Eprint Archive*.
- [41] Ren Pei Xin, Gu Xiao Zhuo. **2022**. Practical Post-quantum Password-Authenticated Key Exchange Based-on Module-Lattice. *International Conference on Information Security and Cryptology*, 137-156.
- [42] Guo Song Hui, Song Yun Fan, Guo Song *et al.* **2023**. Three-Party Password Authentication and Key Exchange Protocol Based on MLWE. *Symmetry*, 15(9): 1750.